

Using Generative Methods for Motion Planning

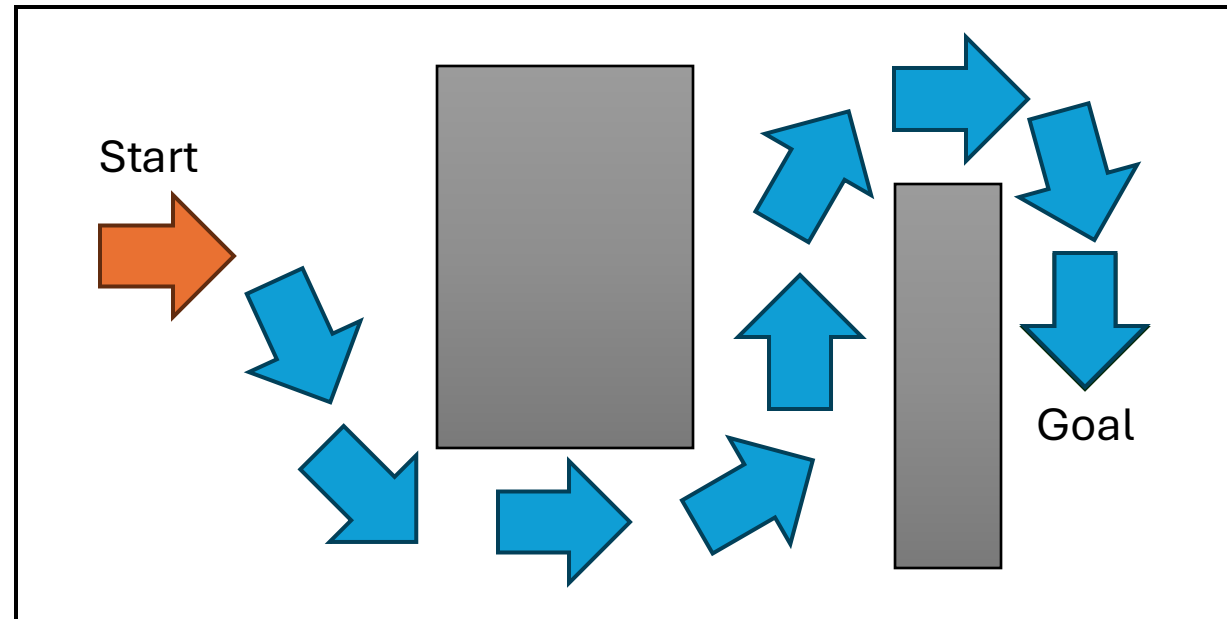
Saad Muhammad Abdul Ghani

Outline

- Prelim - Motion Planning
- Prelim - Generative Modeling
 - VAE, Diffusion
 - Generative Methods
- Taxonomy
- Select Papers
- Open Problems

Preliminary – Motion Planning

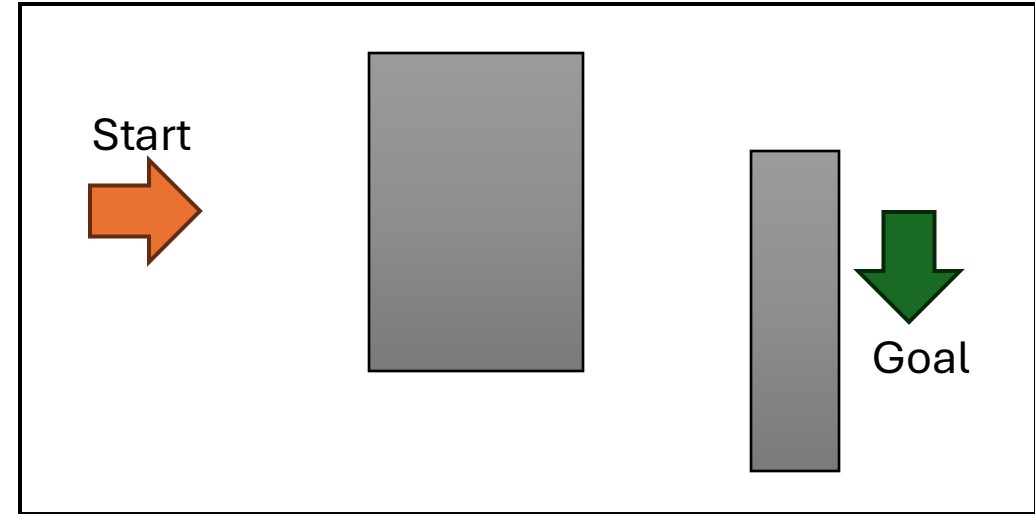
Motion Planning



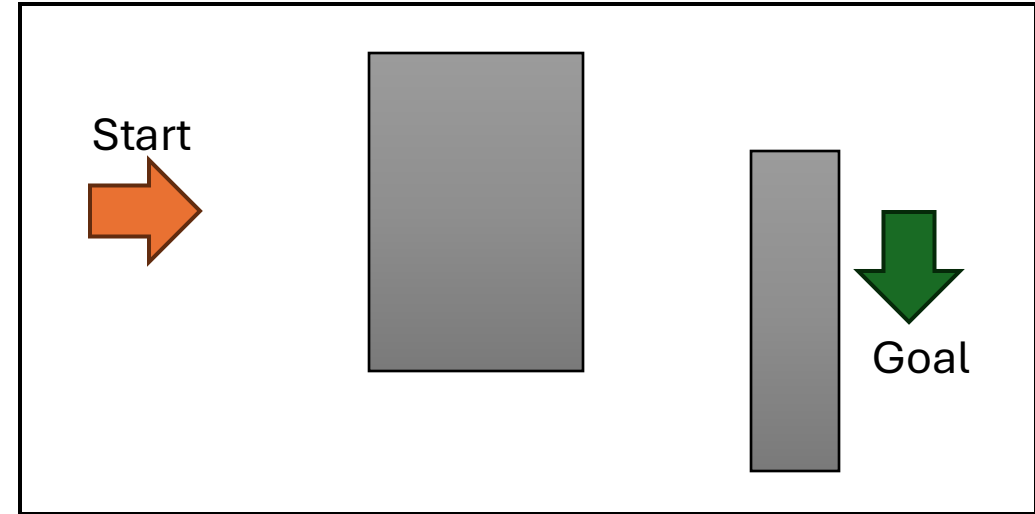
$$Q = Q_{free} \cup Q_{obst}$$

$$p^* = f(Q_{free} | s, g)$$

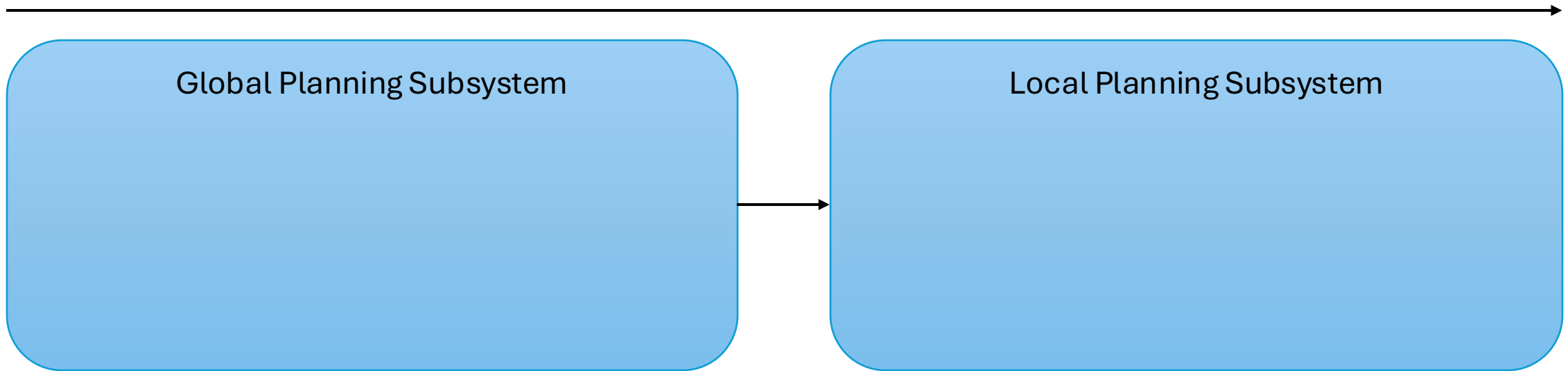
Planning Pipeline



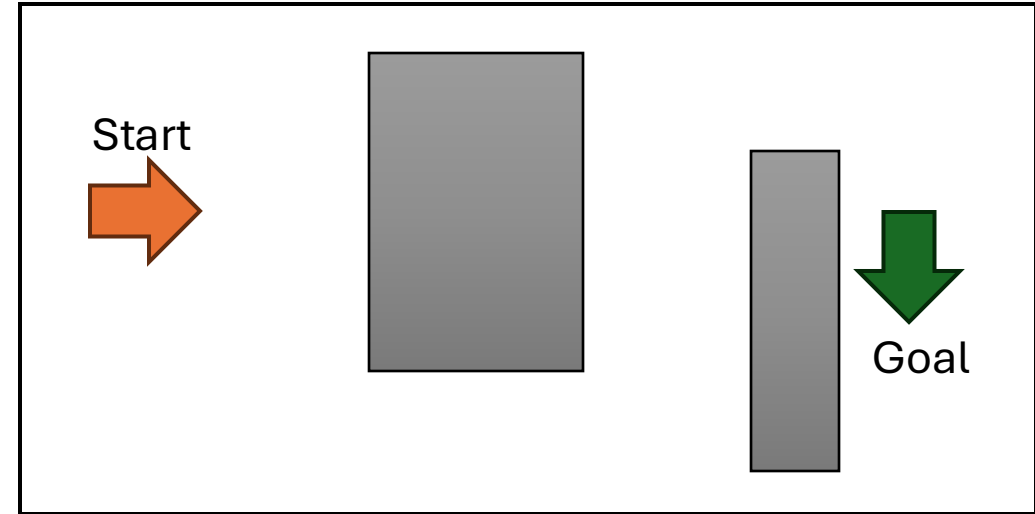
Planning Pipeline



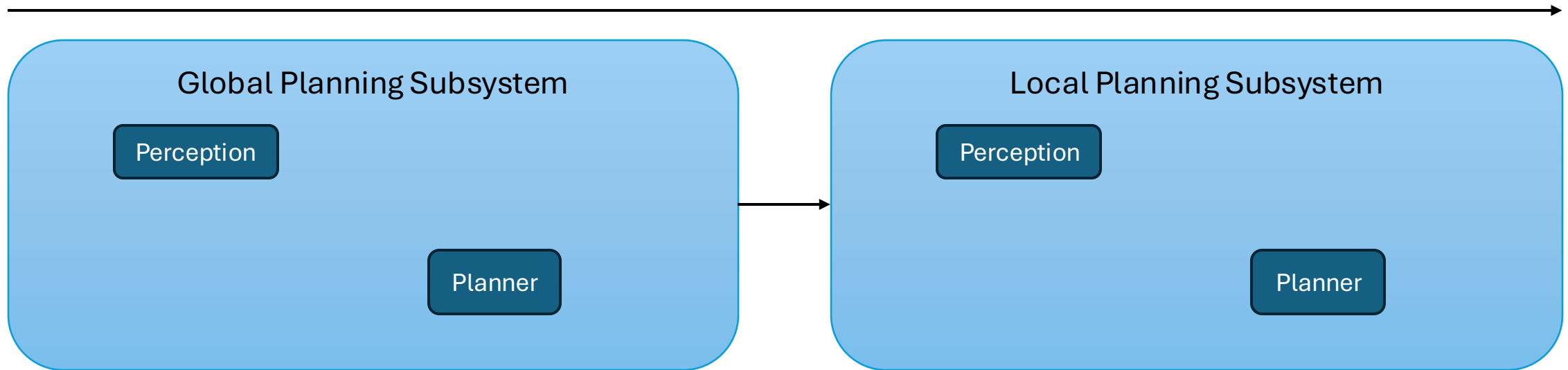
Navigation Pipeline



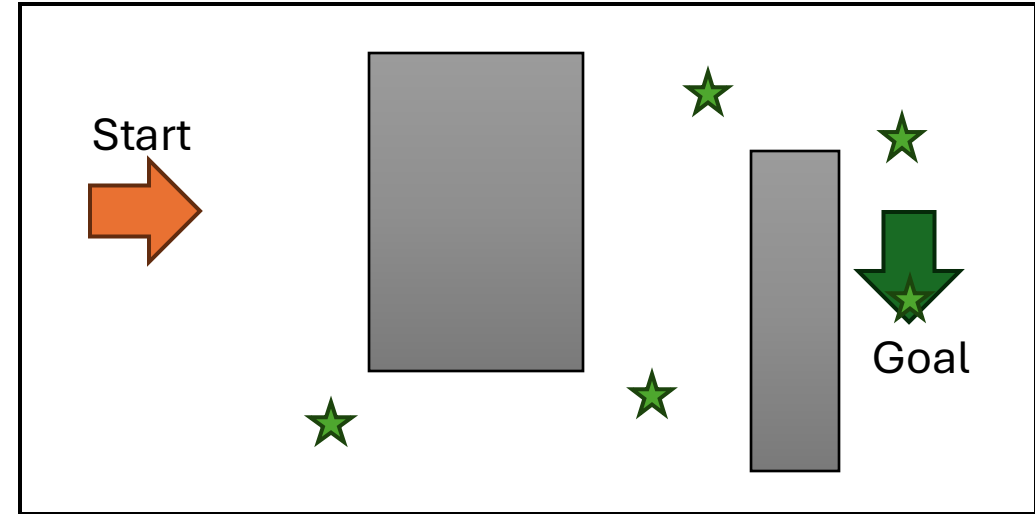
Planning Pipeline



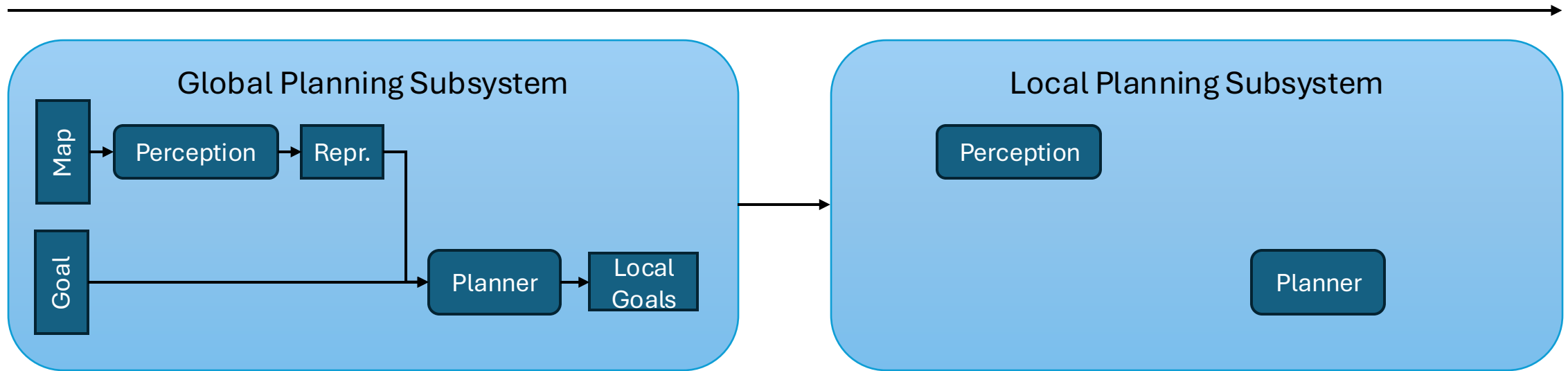
Navigation Pipeline



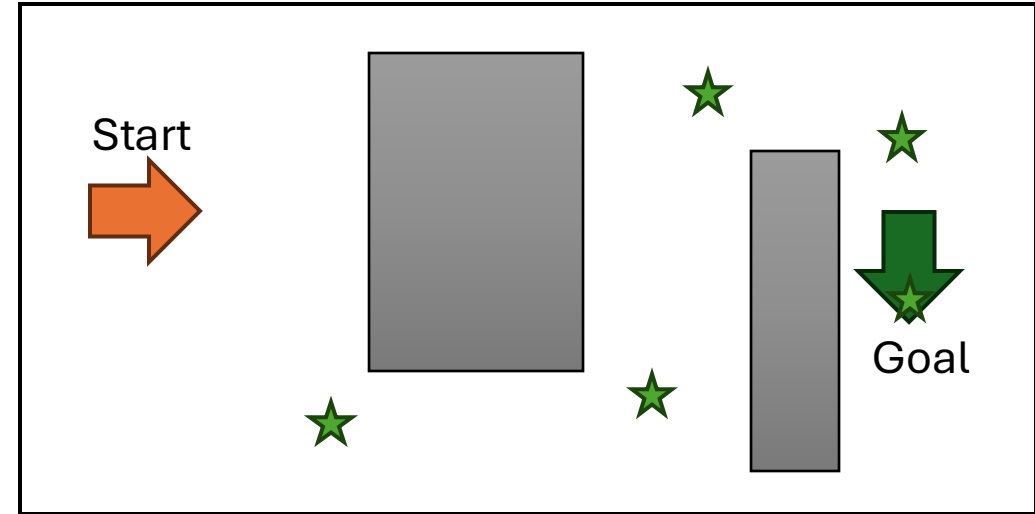
Planning Pipeline



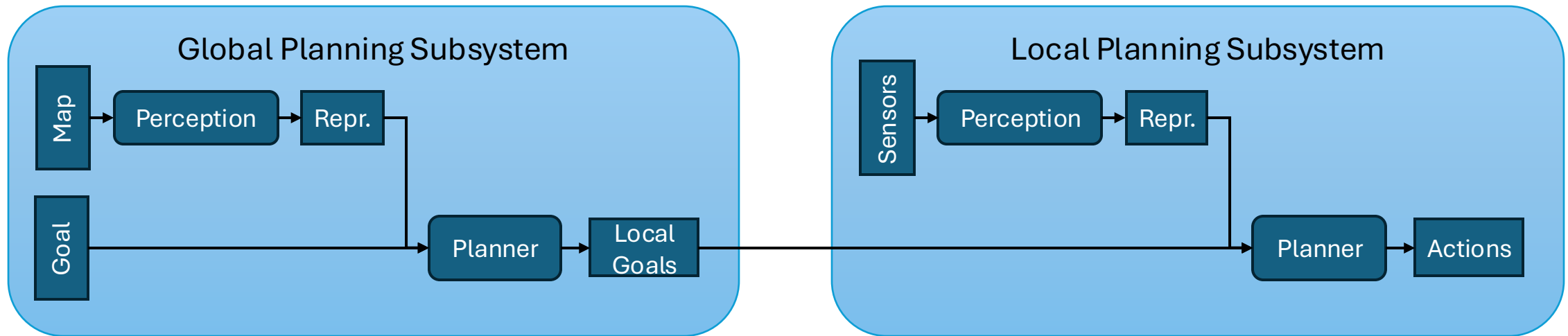
Navigation Pipeline



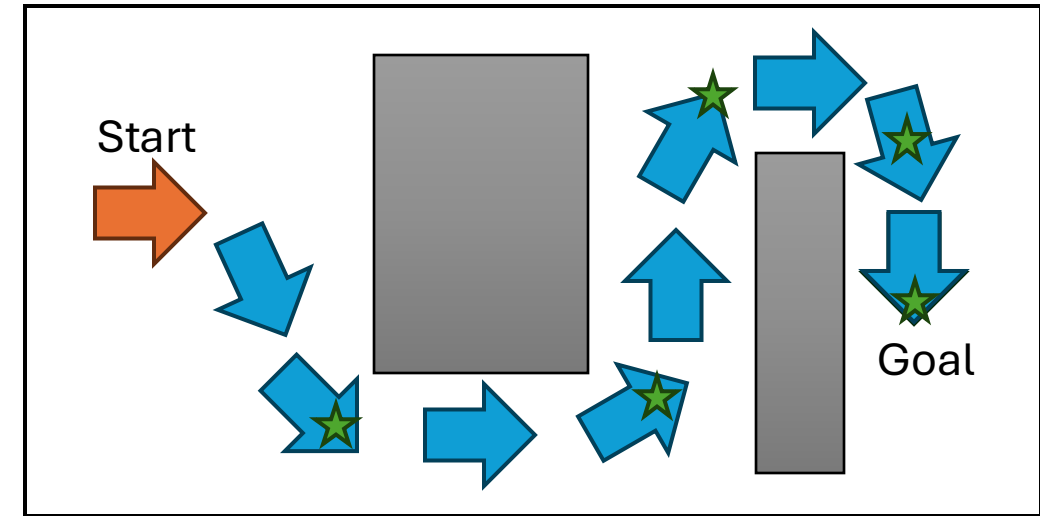
Planning Pipeline



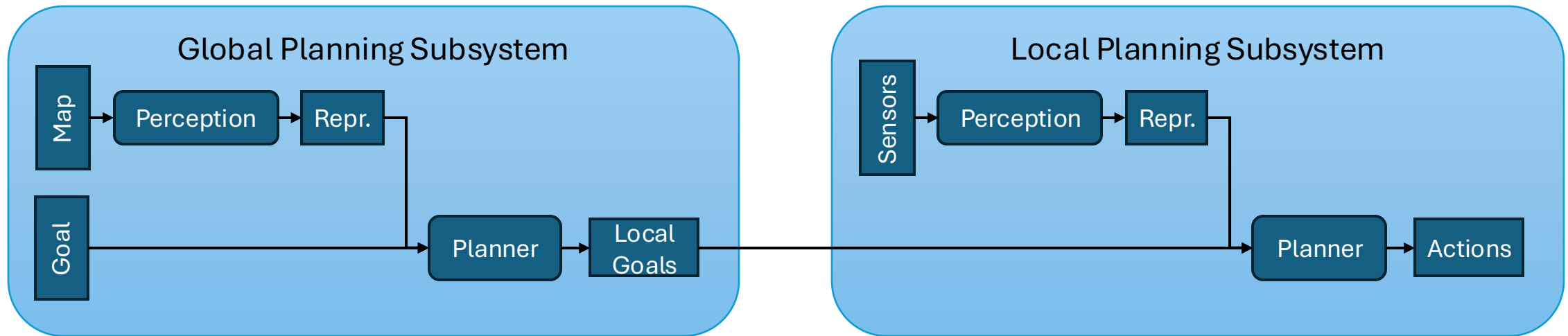
Navigation Pipeline



Planning Pipeline

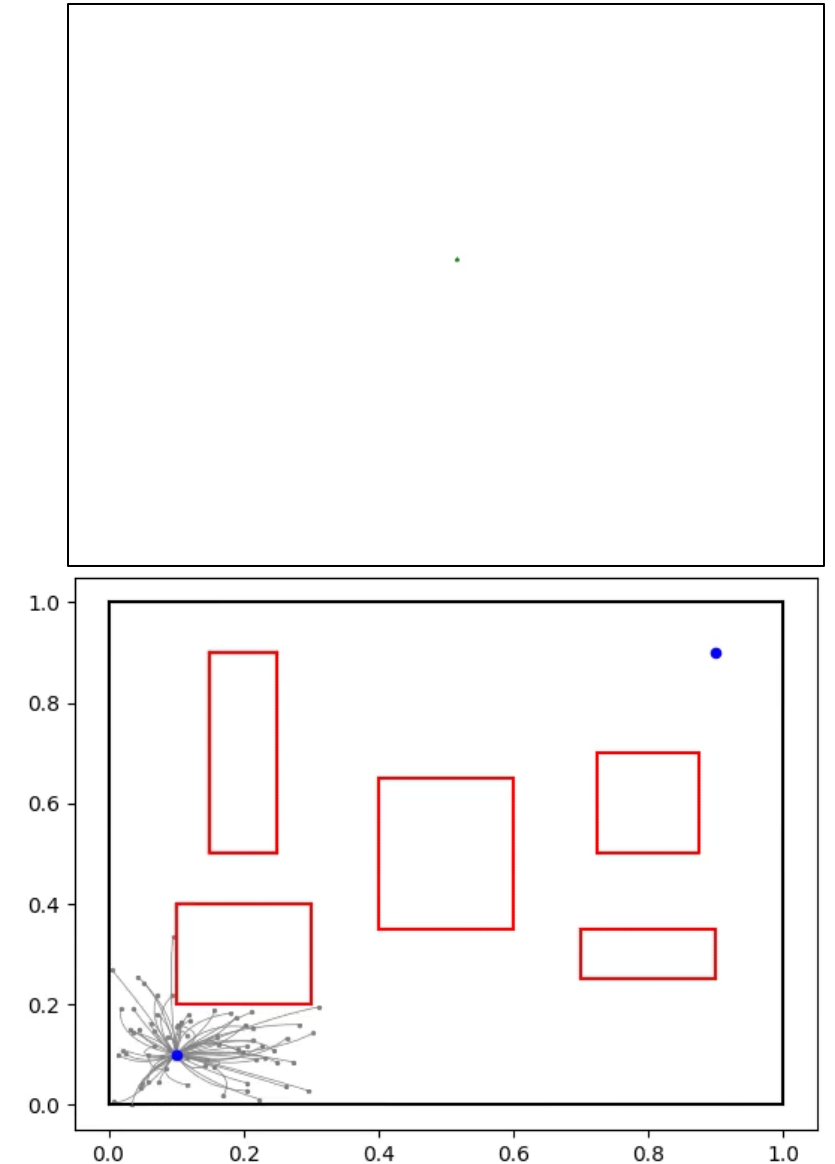


Navigation Pipeline



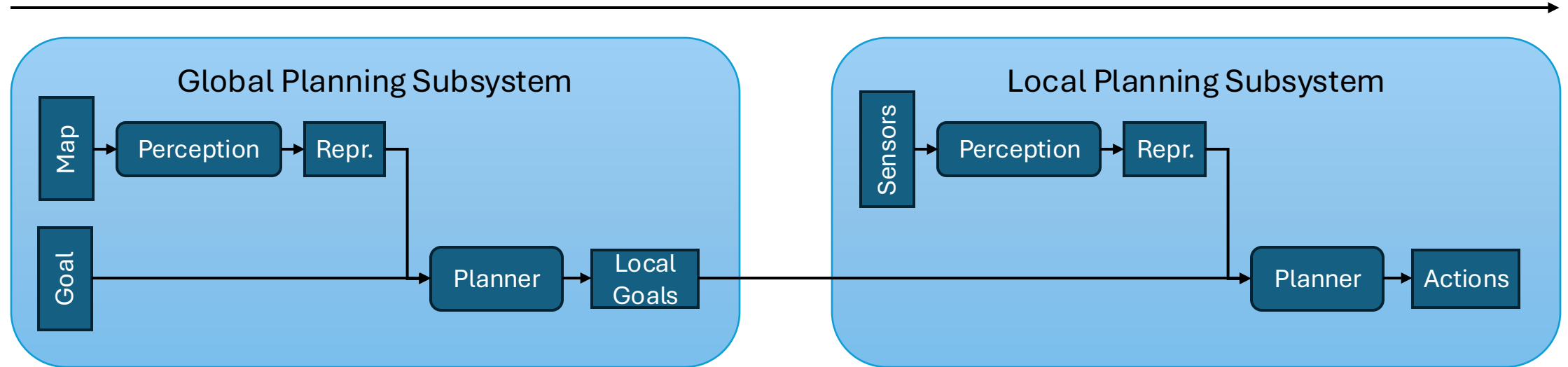
Types of Motion Planners

- Sampling Based Planners
 - Rapidly-exploring Random Tree (RRT)
 - Fast Marching Tree (FMT*)
 - **Samples Uniformly**
- Optimization Based Planners
 - CHOMP
 - **Stuck in local minimas**
 - **Highly dependent on initialization**



Gen. Methods in Planning Pipeline

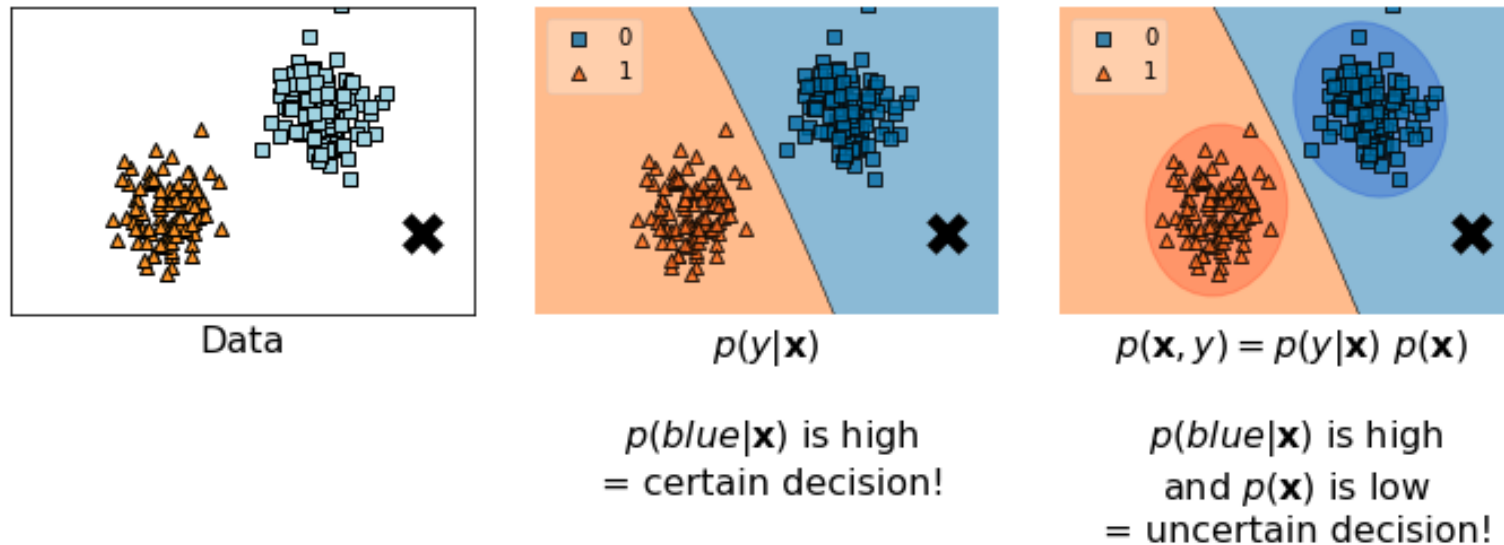
Navigation Pipeline



- Gen. Methods used *purely as planners* – *Diffusion*
- Gen. Methods used as *hybrid planners* – *CVAE + FMT**
- Gen. Methods used to *augment the representation* of env. – *LfLH*

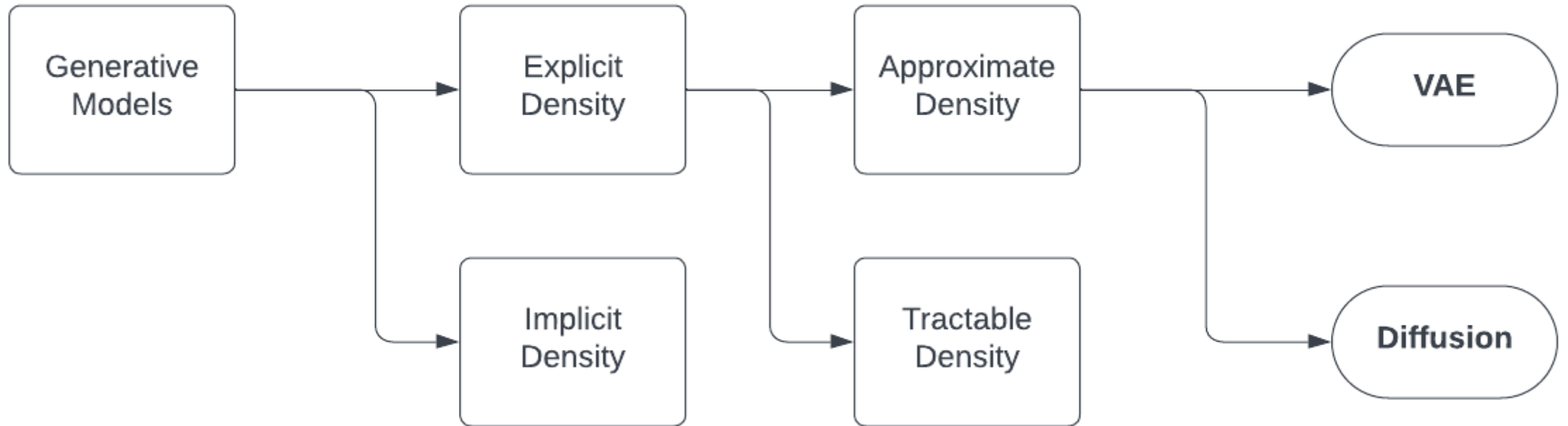
Preliminary - Generative Models and Methods

Generative Models

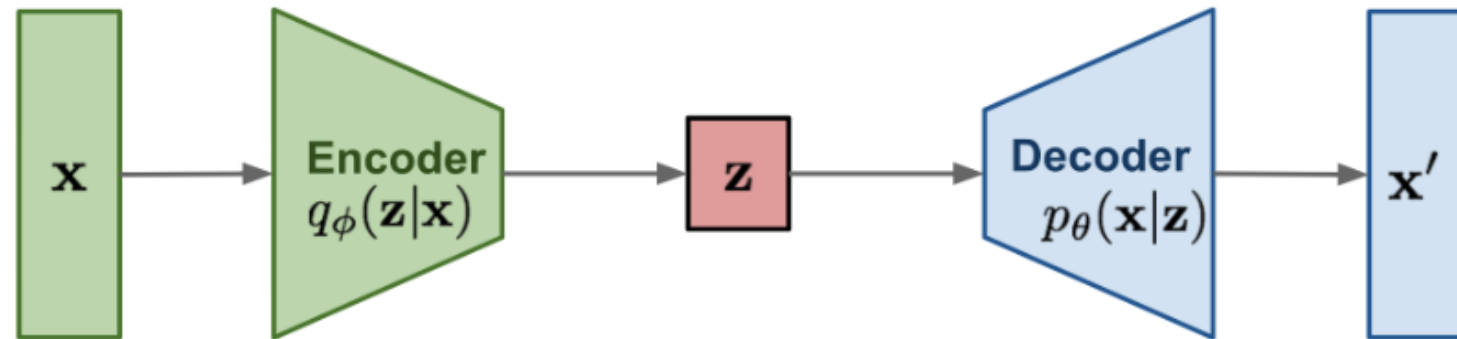


- Discriminative models – $p(y | x)$
- Generative models – $p(y, x)$ or $p(x)$

Generative Models Taxonomy



Generative Models - VAE

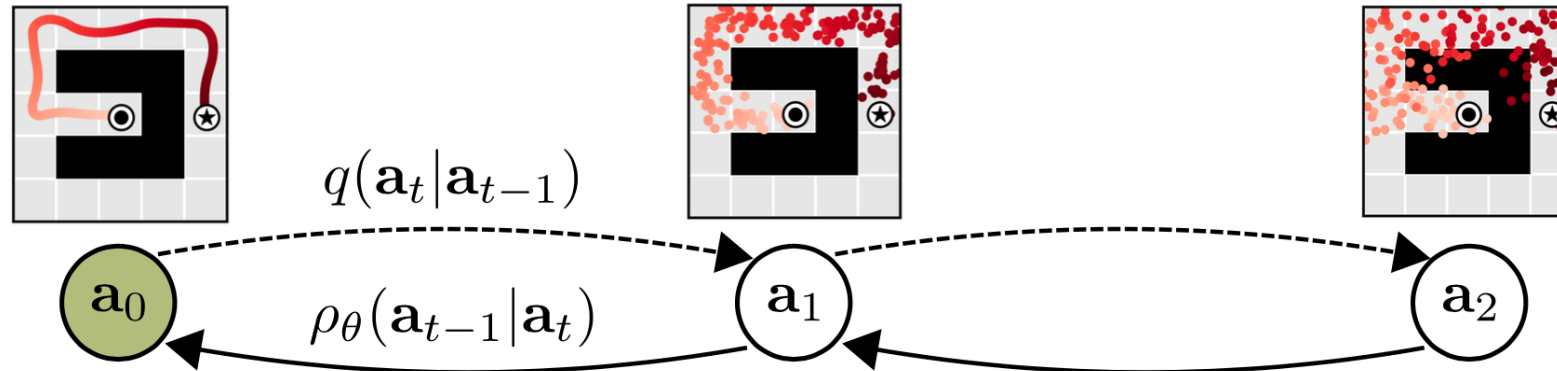


- Model $p(\mathbf{x})$ conditioned on latent var. $\mathbf{z}$:
 $\mathbf{x} \sim p(\mathbf{x}|\mathbf{z}), \quad \mathbf{z} \sim p(\mathbf{z}), \quad p(\mathbf{z}) \sim N(0, \mathbf{I})$
- Training – maximizing lower-bound on log likelihood (ELBO)

$$ELBO = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - KL(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$$

- Conditional VAE – learns $p_{\theta}(\mathbf{x} | \mathbf{z}, \mathbf{y}), q_{\phi}(\mathbf{z} | \mathbf{x}, \mathbf{y})$

Generative Models - Diffusion



- Model $p(\mathbf{x})$ as a Markov chain from noise:

$$p(\mathbf{x}) = p(\mathbf{x}_0) = \int p(\mathbf{x}_N) \prod_{k=1}^N p_\theta(\mathbf{x}_{k-1} | \mathbf{x}_k) d\mathbf{x}_{1:N}, \quad p(\mathbf{x}_N) \sim N(0, \mathbf{I})$$

- If noise added is small, approx. denoising step as a gaussian

$$q(\mathbf{x}_t | \mathbf{x}_0) = N(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad \bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i) \quad p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = N(f(\epsilon_\theta(\mathbf{x}_t, t)), \beta_t \mathbf{I})$$

- Simplified training – minimizing reconstruction loss

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|_2^2], \quad \epsilon \sim N(0, \mathbf{I})$$

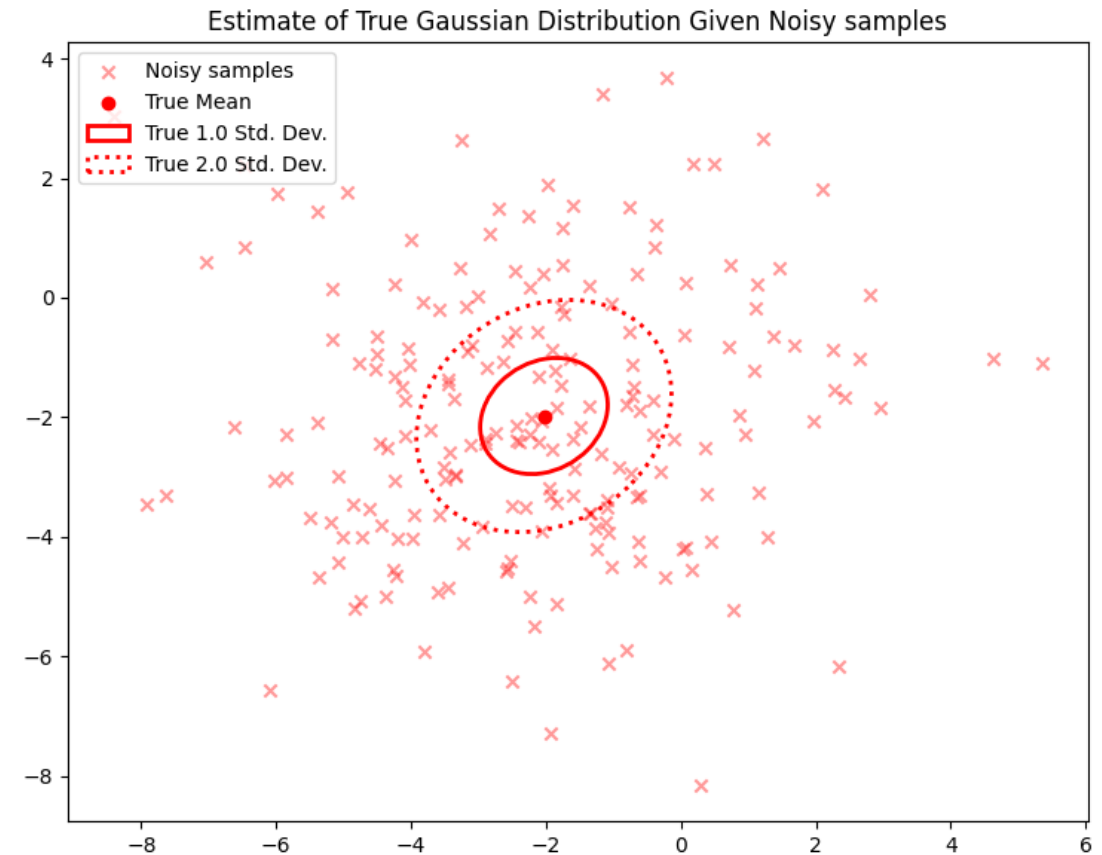
Generative Method – Density Estimation

- Estimates parameters of a density function conditioned on some data

$$p(\mathbf{y}|\mathbf{x}) = N(\mu_{\theta}(\mathbf{x}), \Sigma_{\theta}(\mathbf{x}))$$

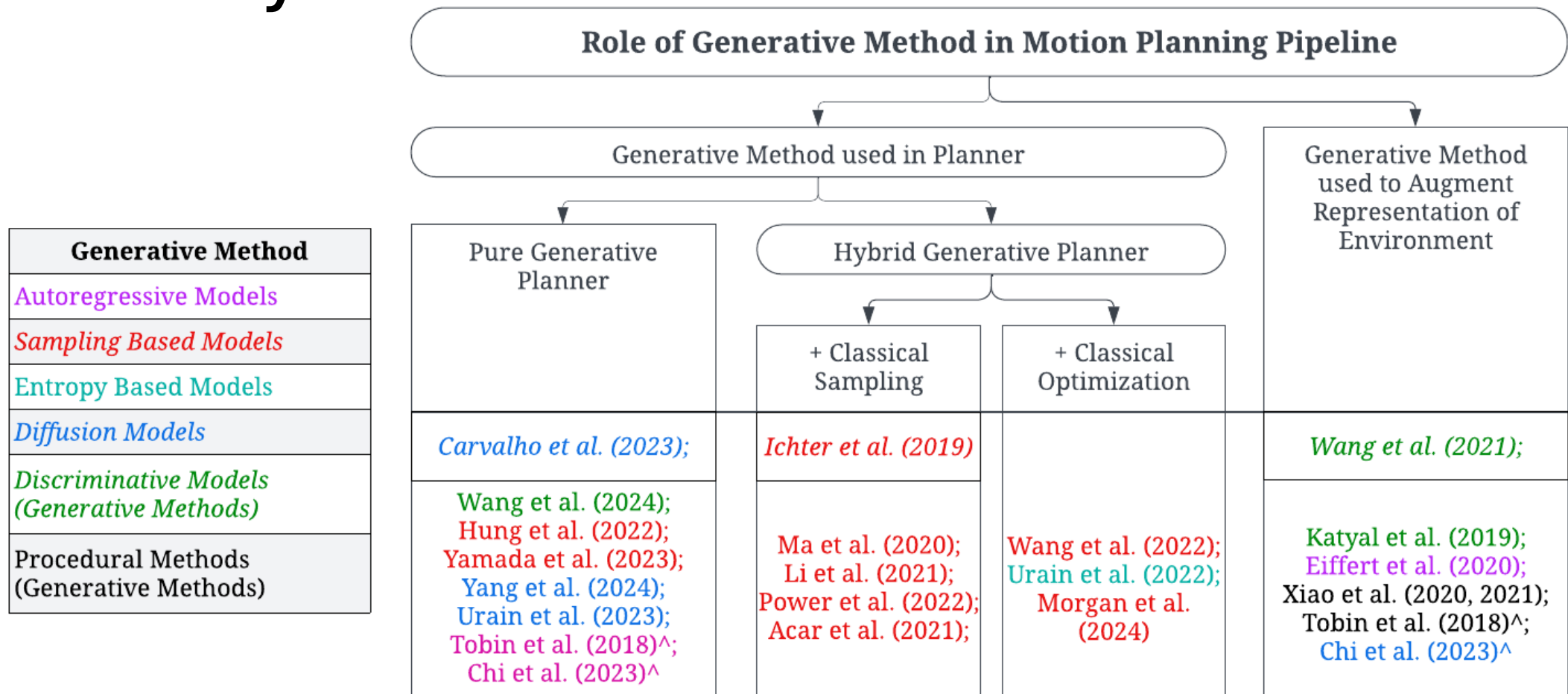
- Parameters are modelled as outputs of NN

$$\mu_{\theta}(\mathbf{x}), \Sigma_{\theta}(\mathbf{x})$$

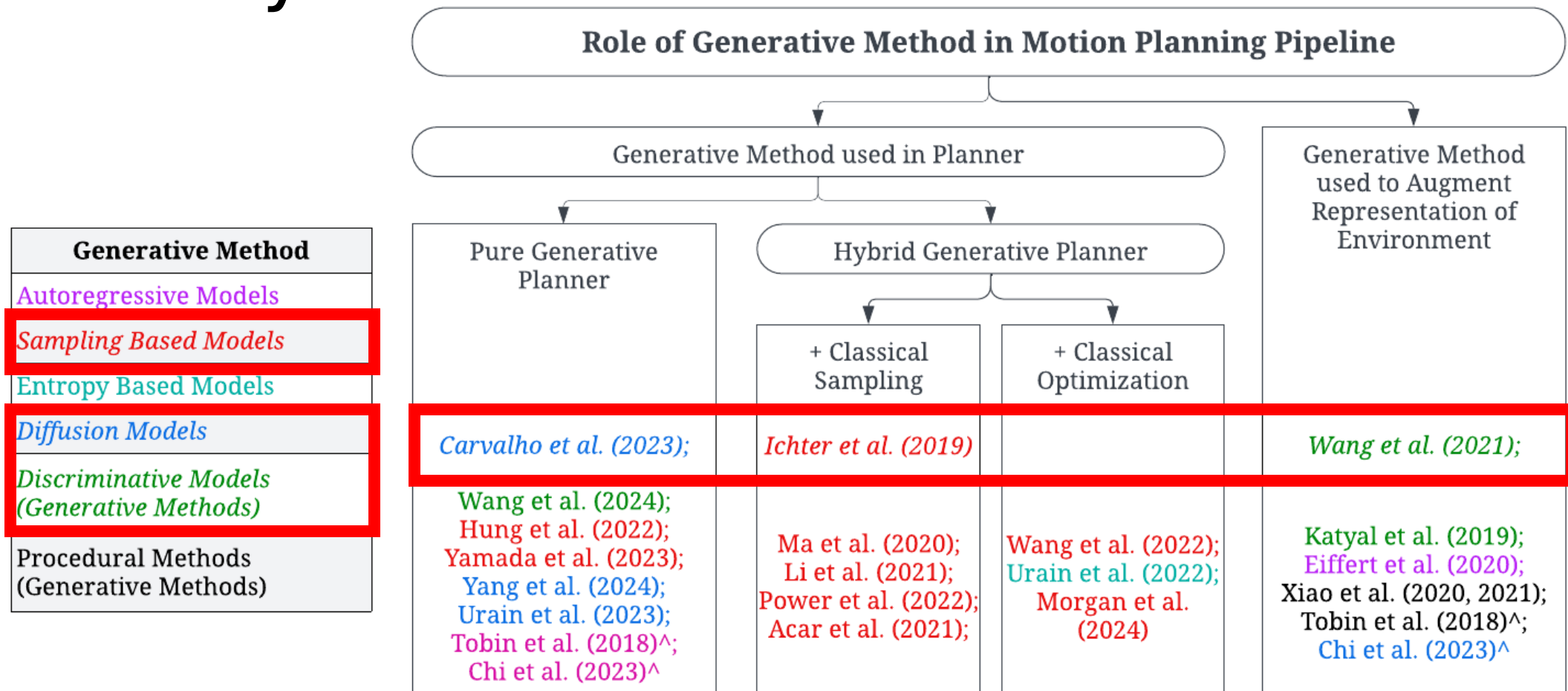


Taxonomy

Taxonomy

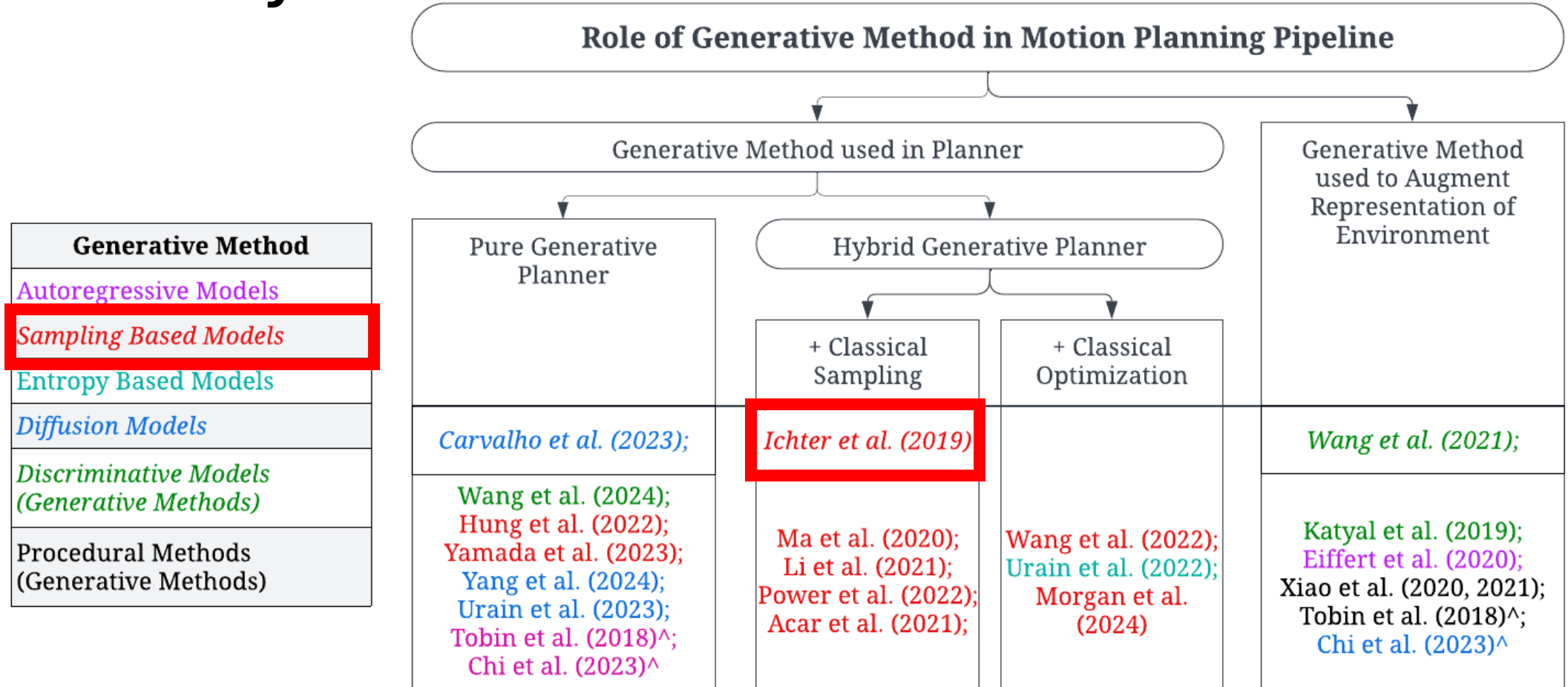


Taxonomy



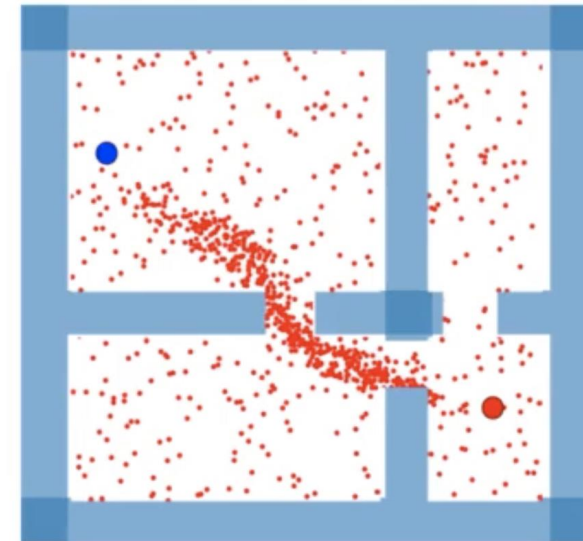
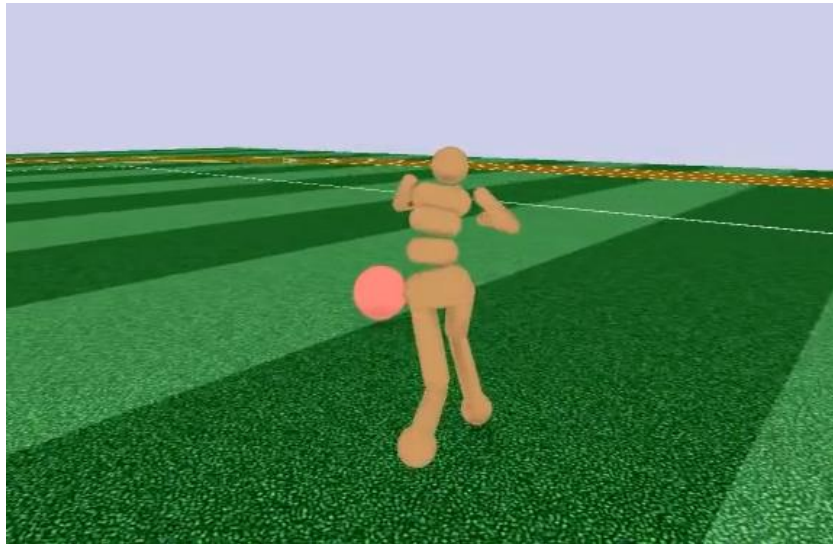
Selected Papers

Taxonomy



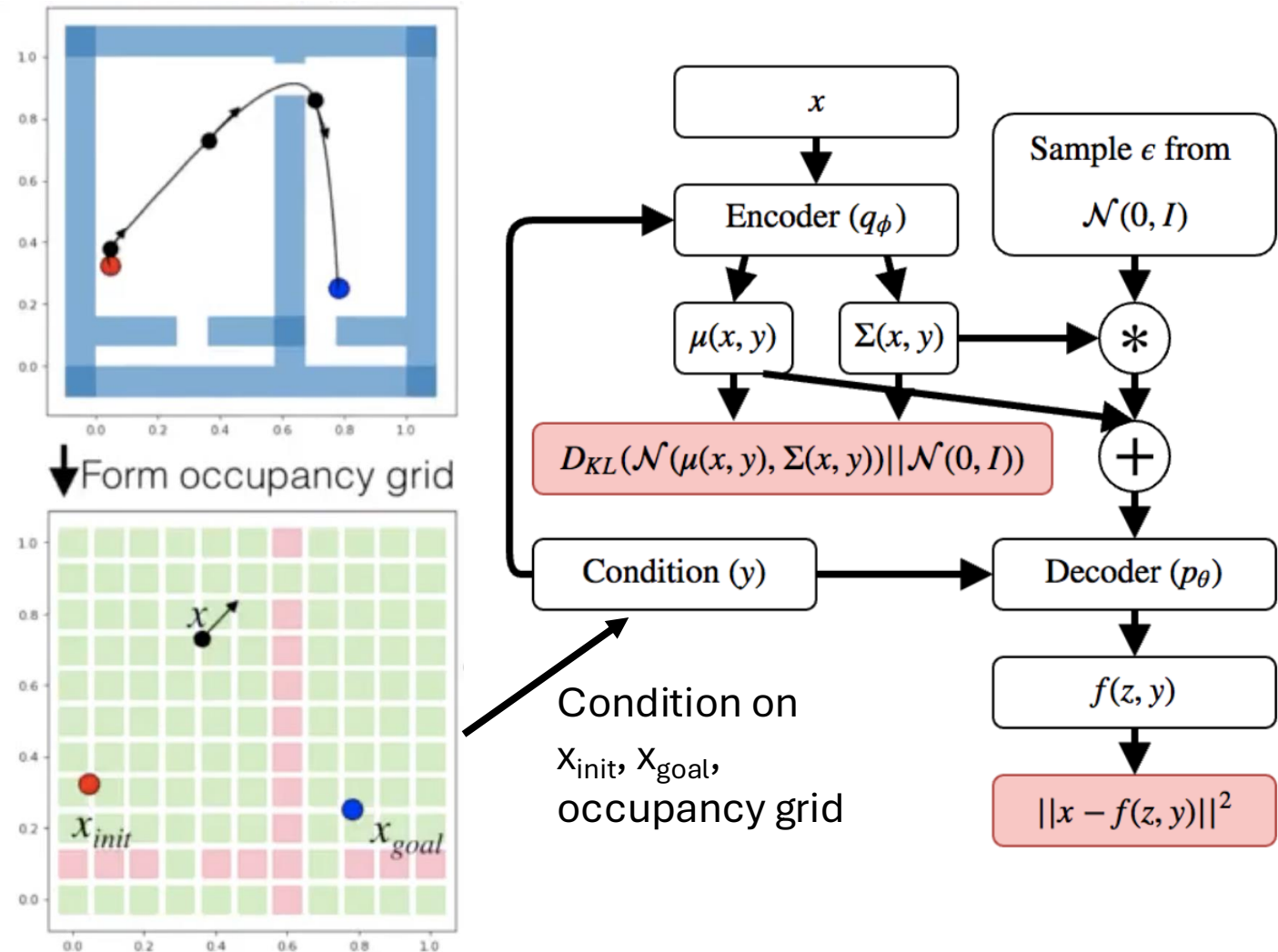
Learning Sampling Distributions for Robot Motion Planning

- Sampling Based Motion Planners assume uniform distribution over configuration space
 - Often the feasible plan is only in a “small” region of the space

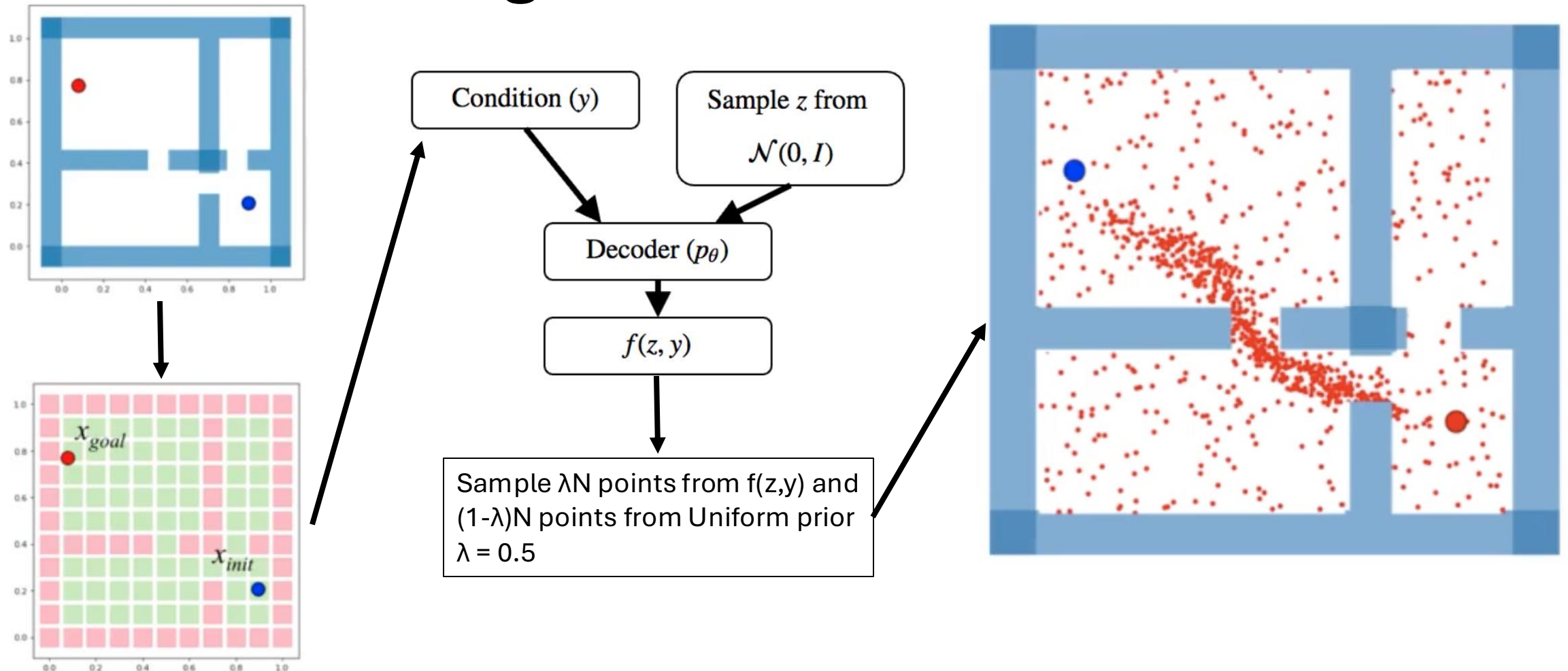


Learning Sampling Distributions for Robot Motion Planning

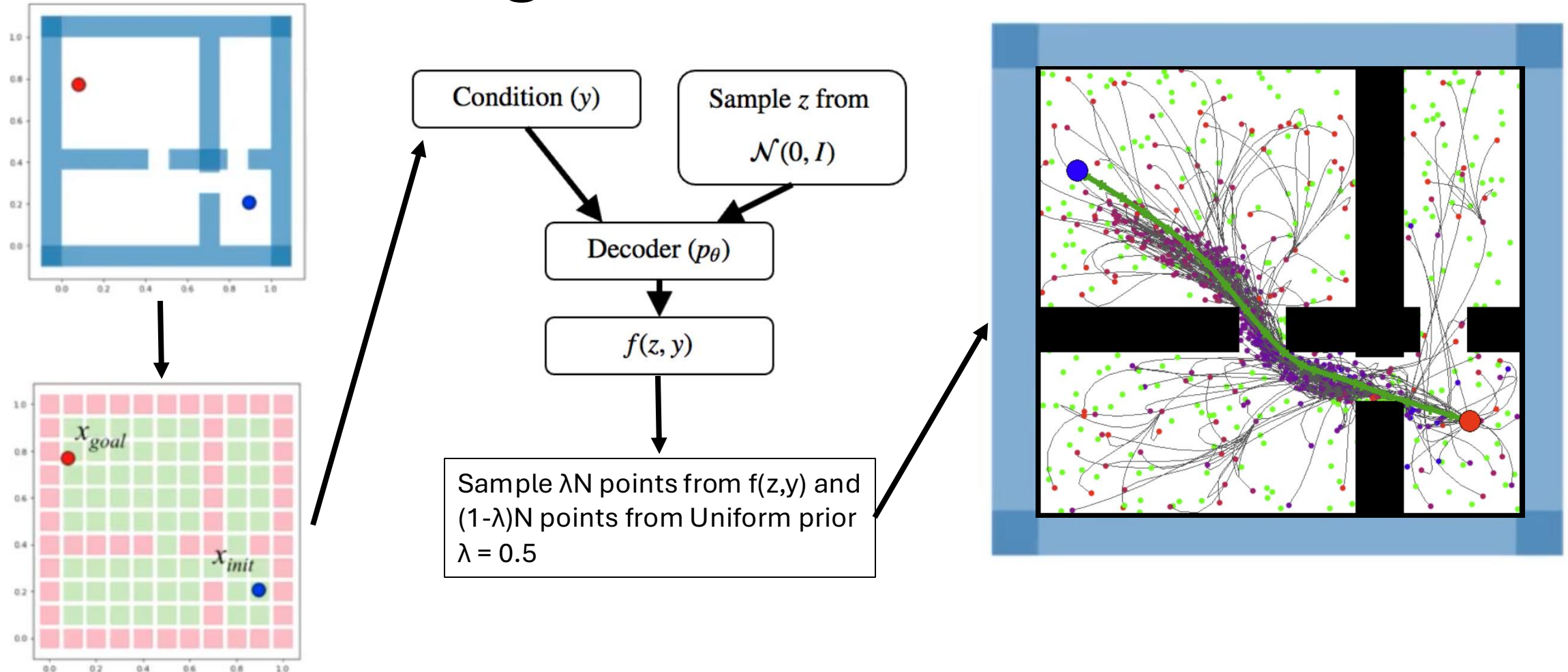
- Learn Sampling Distribution from demonstration to bias sampling
 - Using a Conditional VAE



Learning Sampling Distributions for Robot Motion Planning



Learning Sampling Distributions for Robot Motion Planning

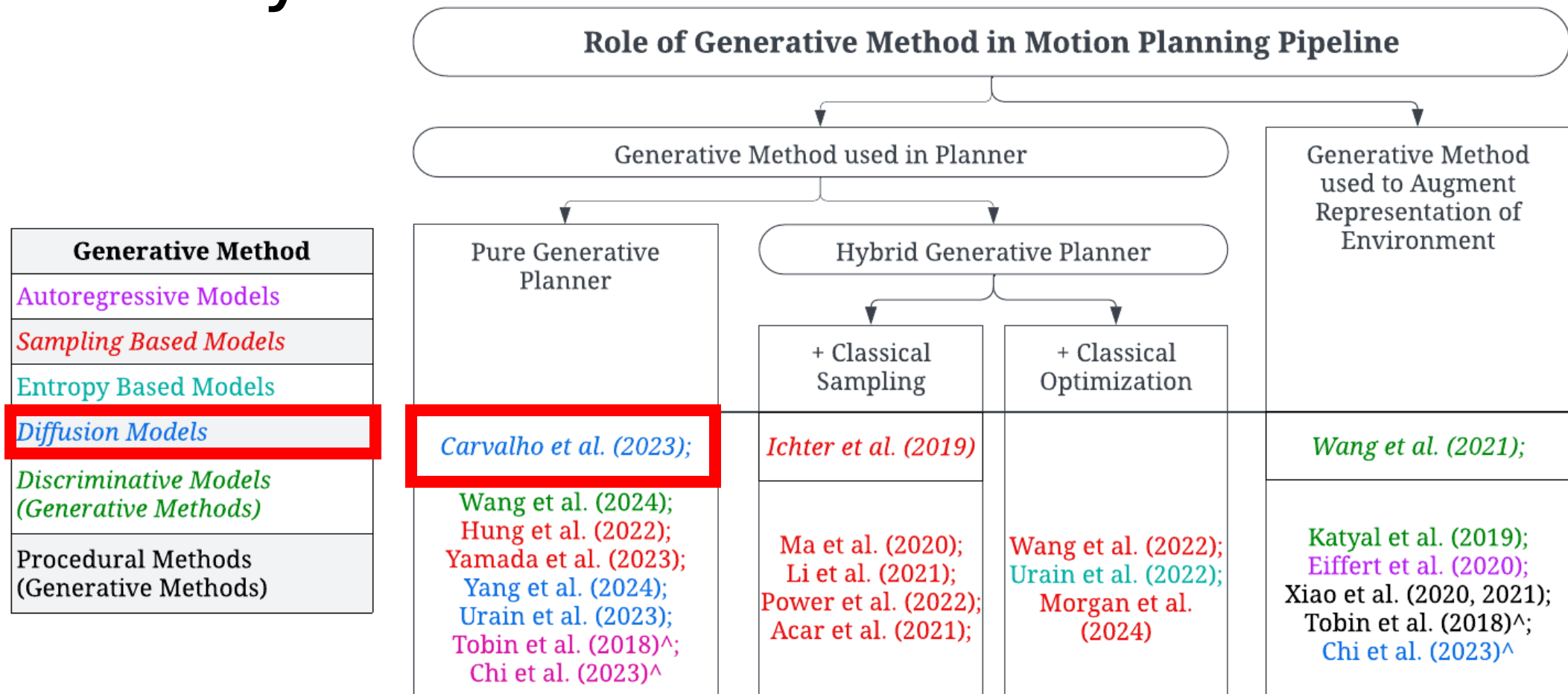


Learning Sampling Distributions for Robot Motion Planning

- Advantages:
 - Combines the asymptotic guarantees of classical sampling techniques with the speed of using a learned distribution
- Problems:
 - Training on $\sim 100k$ plans, requires gathering of demonstrations
 - CVAE models $p(x | y)$, if

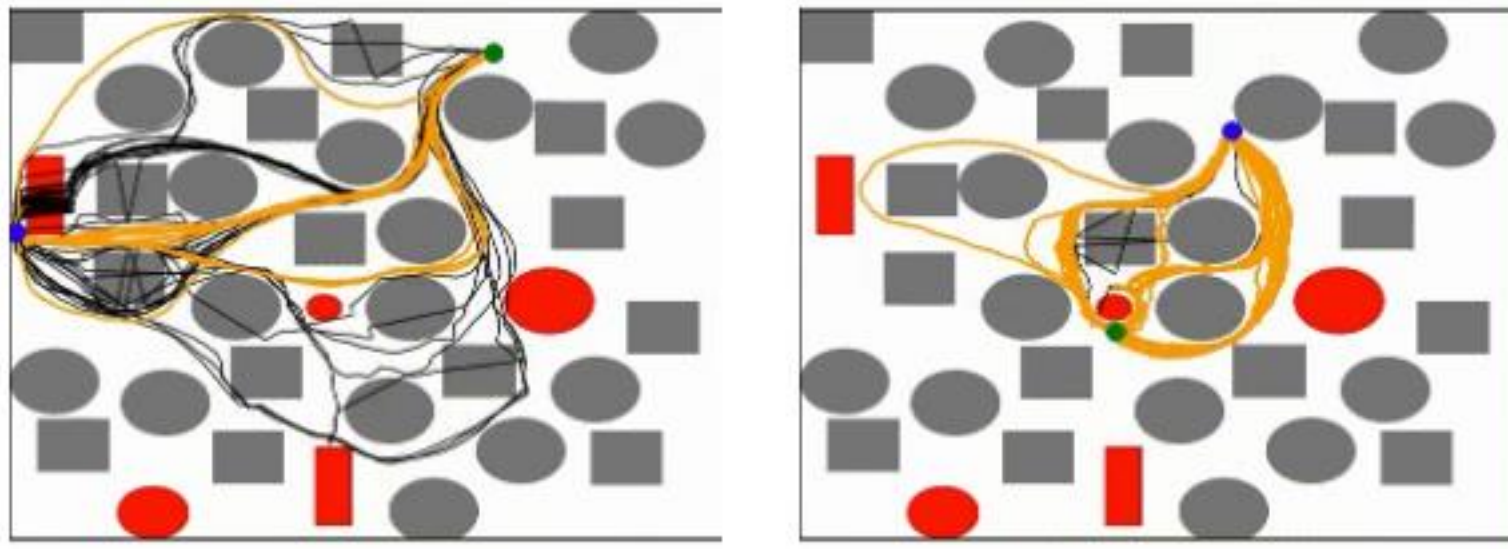


Taxonomy



Motion Planning Diffusion: Learning and Planning of Robot Motions with Diffusion Models

- Attempts to model $p(x)$ only as a prior
 - Uses Diffusion model
- Guidance is used during denoising process to optimize the trajectory to task objectives, in a planning-as-inference way



Motion Planning Diffusion: Learning and Planning of Robot Motions with Diffusion Models

- Motion Planning Optimization

$$\tau^* = \arg \min_{\tau} \sum_i \lambda_i c_i(\tau)$$

- MP Optimization as inference

$$p(\tau|\mathcal{O}) \propto p(\mathcal{O}|\tau)p(\tau)$$

$$p(\mathcal{O}|\tau) \propto \prod_i p_i(o_i|\tau)^{\lambda_i}$$

$$p_i(o_i|\tau) \propto \exp(-c_i(\tau))$$

Symbol	Description
τ	$(s_0, \dots, s_{H-1}) \in \mathbb{R}^{H \times d}$
H	Time Horizon
d	Dimension of state space
$c_i(\tau)$	Cost terms (e.g., collision, joint limits, smoothness)
$\mathcal{O}$	Task objectives (e.g., obstacles, start and goal states)

$$p(\tau|\mathcal{O}) \propto \prod_i \exp(-\lambda_i c_i(\tau))p(\tau)$$

$$\tau^* = \arg \min_{\tau} \sum_i \lambda_i c_i(\tau) - \log p(\tau)$$

Motion Planning Diffusion: Learning and Planning of Robot Motions with Diffusion Models

- Diffusion Model

$$\underline{p_{\theta}(\tau_{t-1}|\tau_t, t)} = N(\mu_t = f(\epsilon_{\theta}(\tau_t, t)), \Sigma_t = \beta_t \mathbf{I})$$

Symbol	Description
τ	$(s_0, \dots, s_{H-1}) \in \mathbb{R}^{H \times d}$
H	Time Horizon
d	Dimension of state space
$c_i(\tau)$	Cost terms (e.g., collision, joint limits, smoothness)
$\mathcal{O}$	Task objectives (e.g., obstacles, start and goal states)

- Optimization with Guidance from Task Objectives

$$p(\tau_0|\mathcal{O}) = p(\tau_N|\mathcal{O}) \prod_{t=1}^N p(\tau_{t-1}|\tau_t, t, \mathcal{O})$$

$$p(\tau_{t-1}|\tau_t, t, \mathcal{O}) \propto \underline{p(\tau_{t-1}|\tau_t, t)} \underline{p(\mathcal{O}|\tau_{t-1})}$$

Motion Planning Diffusion: Learning and Planning of Robot Motions with Diffusion Models

$$p_{\theta}(\tau_{t-1}|\tau_t, t) = N(\underbrace{\mu_t = f(\epsilon_{\theta}(\tau_t, t))}_{\text{MP optimization posterior}}, \Sigma_t = \beta_t \mathbf{I}) \quad p(\tau_{t-1}|\tau_t, t, \mathcal{O}) \propto \underbrace{p(\tau_{t-1}|\tau_t, t)}_{\text{diffusion prior}} \underbrace{p(\mathcal{O}|\tau_{t-1})}_{\text{Objectives Likelihood}}$$

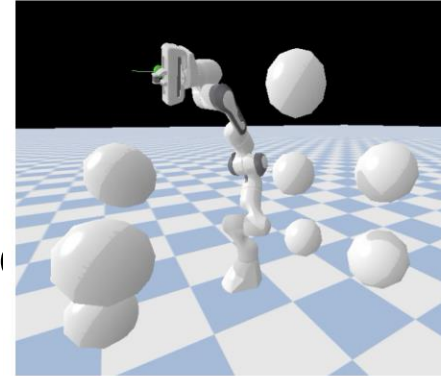
- Posterior cannot be sampled from in closed-form
 - Taking logarithm, $\Sigma_t \rightarrow 0$ as $t \rightarrow 0$, denoising model concentrates mass around mean.
 - Task log-likelihood is approximated with first-order taylor expansion around mean

$$\log p(\tau_{t-1}|\tau_t, t, \mathcal{O}) \propto \log p(\tau_{t-1}|\tau_t, t) p(\mathcal{O}|\tau_{t-1}) \approx \log N(\mu_t + \Sigma_t \mathbf{g}, \Sigma_t)$$

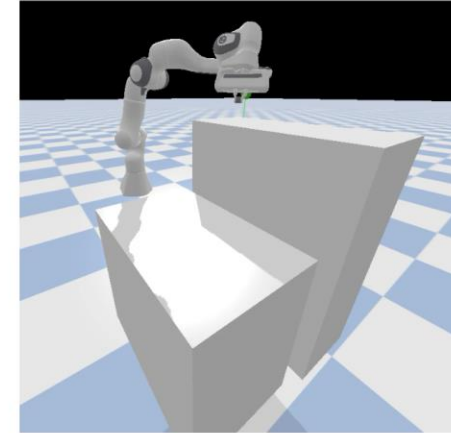
$$\mathbf{g} = \nabla_{\tau_{t-1}} \log p(\mathcal{O}|\tau_{t-1}) = -\sum_i \lambda_i \nabla_{\tau_{t-1}} \underbrace{c_i(\tau_{t-1})}_{\text{cost function}}$$

Motion Planning Diffusion: Learning and Planning of Robot Motions with Diffusion Models

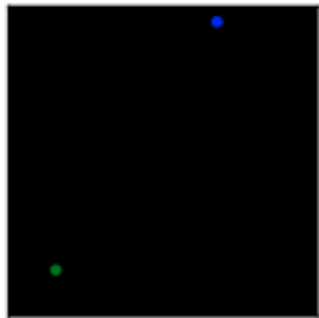
- Advantages:
 - Models $p(x)$ and $p(y|x)$ separately
 - Contextual costs can be added to increase
 - Multi-modal



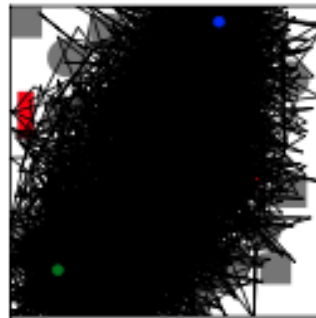
(c) Panda Spheres



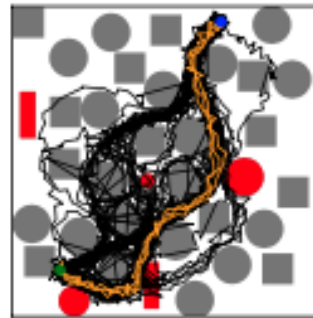
(d) Panda Shelf



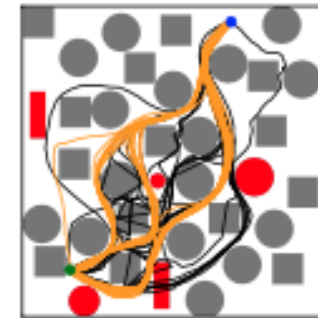
(a) Diffusion Step 0



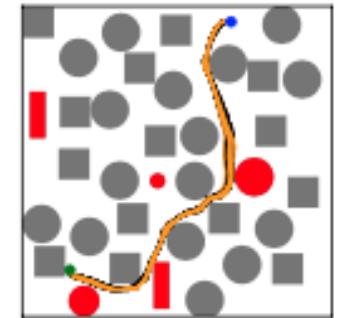
(b) Diffusion Step 10



(c) Diffusion Step 20

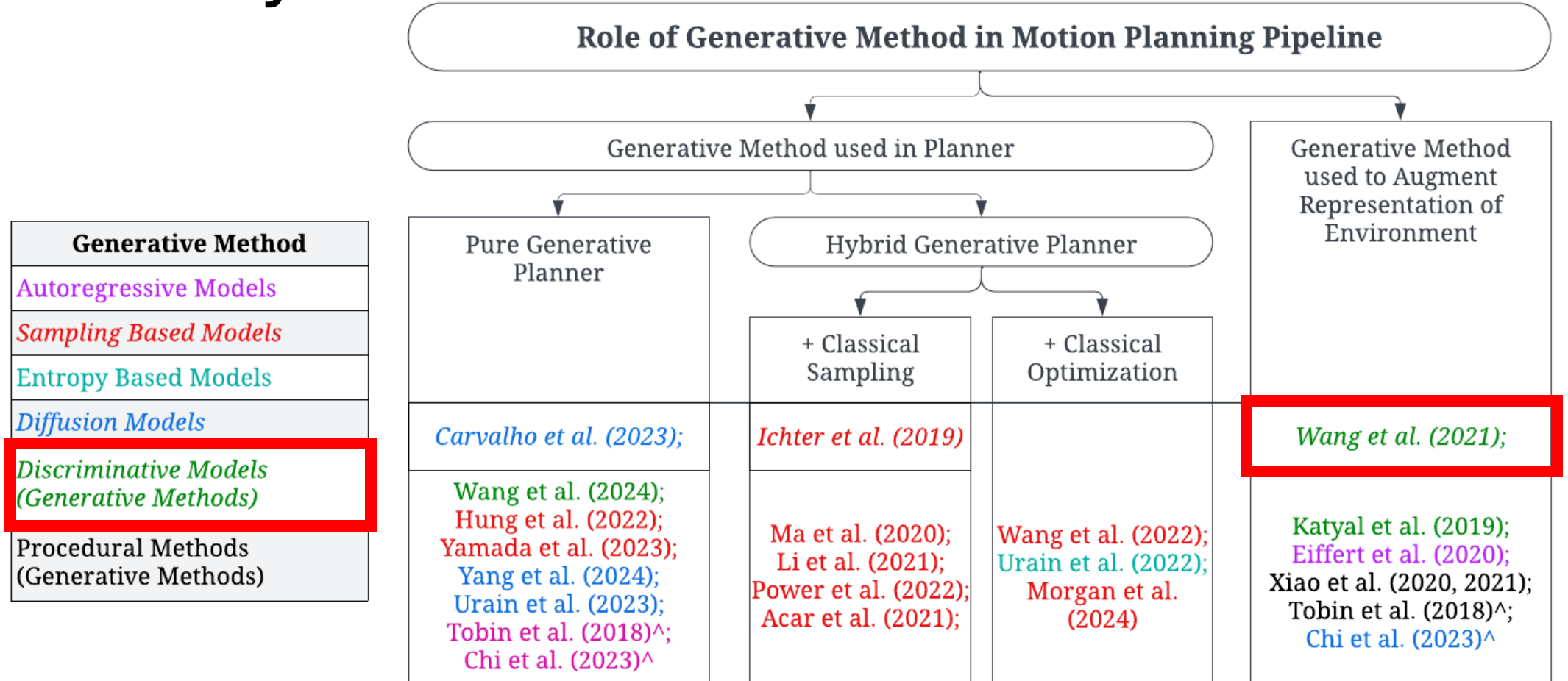


(d) Diffusion Step 25



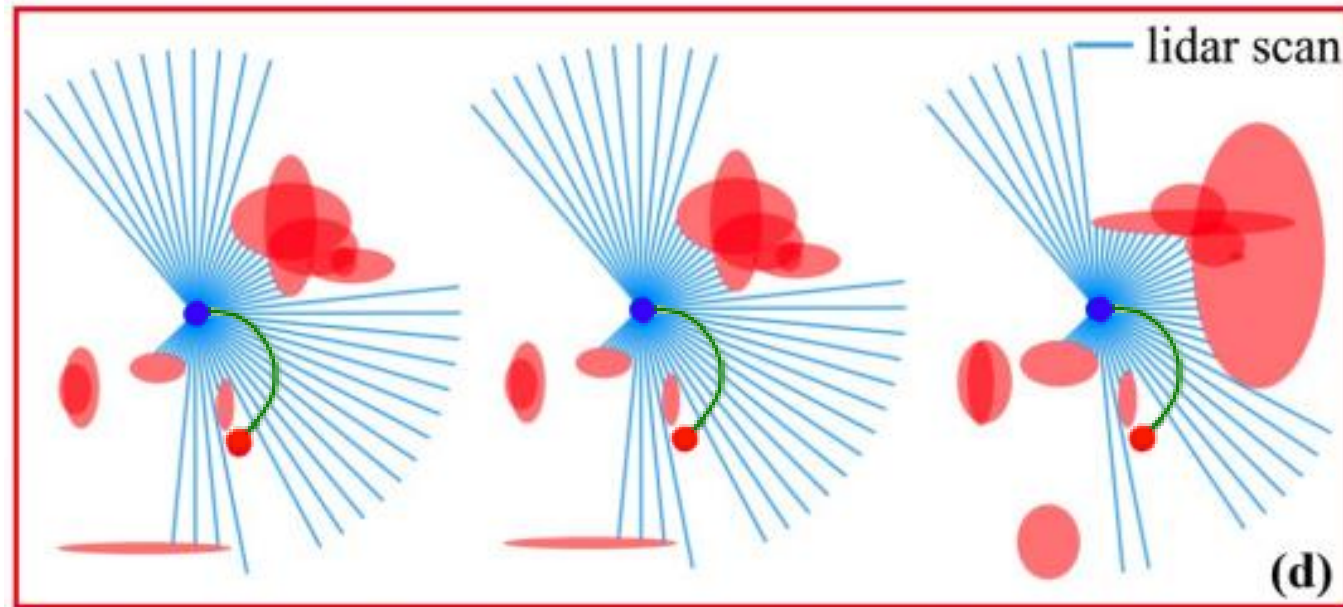
(e) CVAE

Taxonomy



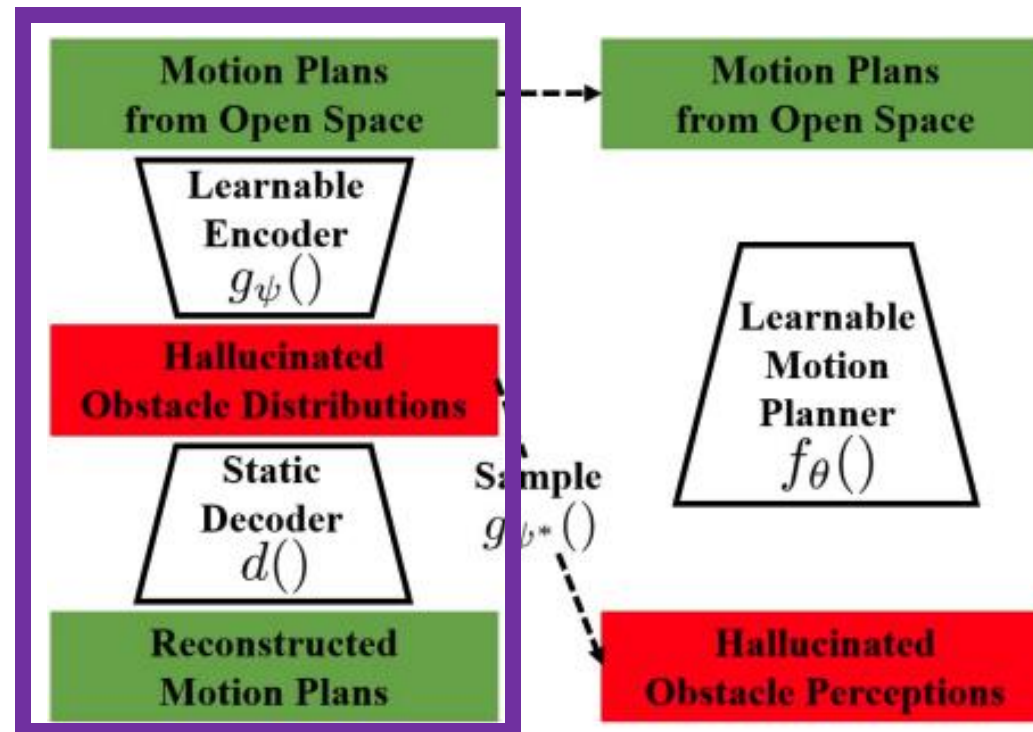
From Agile Ground to Aerial Navigation: Learning from Learned Hallucination (LfLH)

- Expert demonstrations collected in self-supervised approach
- Attempts to learn the ‘inverse’ motion planning problem
 - Given plan, what are all the obstacles it can avoid?



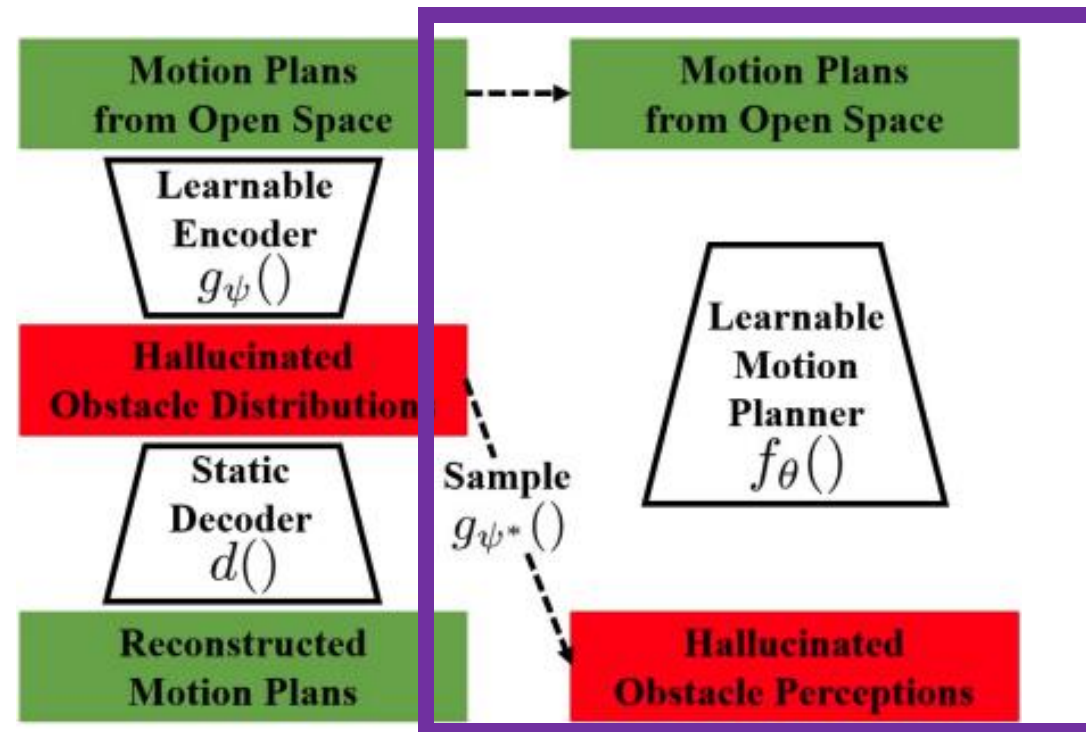
From Agile Ground to Aerial Navigation: Learning from Learned Hallucination (LfLH)

- Learned latent space represents ‘hallucinated’ (imagined) obstacles that make given plan optimal



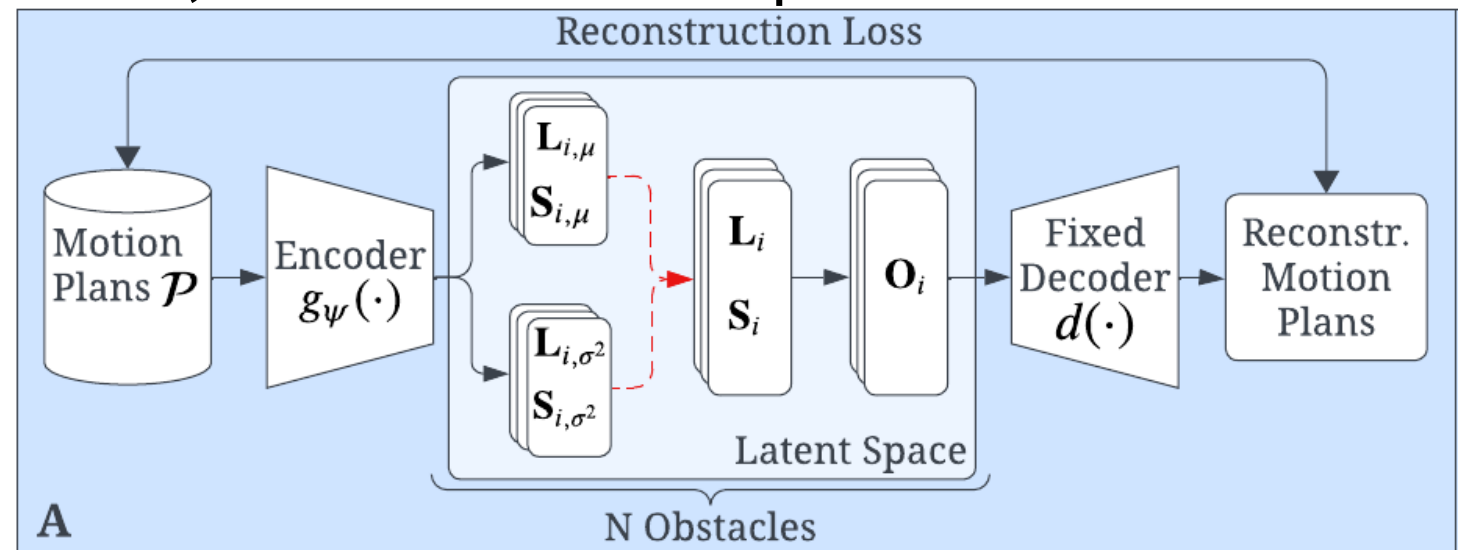
From Agile Ground to Aerial Navigation: Learning from Learned Hallucination (LfLH)

- Sample obstacles from latent space + original plan trains separate motion planning model



From Agile Ground to Aerial Navigation: Learning from Learned Hallucination (LfLH)

- The encoder is a CNN with Autoregressive layers that outputs parameters of N gaussian distributions
 - N = number of hallucinated obstacles
 - Each obstacle is an ellipse, centered at L with size S
- Decoder is a non-learnable, classical motion planner with differentiable layers
- Loss terms:
 - Reconstruction loss
 - Regularizing terms



From Agile Ground to Aerial Navigation: Learning from Learned Hallucination (LfLH)

- Advantages:
 - Hallucination function acts as a generator, learning obstacles given trajectories - $p(y | x)$
 - Can generate large amounts of expert demonstrations that can be used to train planners
- Problems:
 - Assumption on obstacle shape – can it deal with concave shapes?
 - Assumption on static obstacle
 - Training data needs to be comprehensive
 - How diverse is hallucinated obstacles? Mode Collapse?

Open Problems

Open Problems - Generalizability

- Ichter, et al. 2019
 - CVAE – $p(x | y)$
 - Changes to y would greatly affect performance
- Carvalho, et al. 2023
 - Diffusion – $p(x)$
 - Seemingly this would be enough but for same robot (7-dof Franka Robot) in 2 different environments, 10k trajectories were gathered each.
Generalizable?
- Wang, et al. 2021
 - Density Estimation – $p(y | x)$
 - Training data has to be comprehensive in action space
 - Assumptions about obstacles limit applicability

Open Problems - Generalizability

Possible methods:

- ❖ Reasoning about coverage of training data
 - ❖ Collect samples that fulfill coverage.
- ❖ Project OOD data to in-distribution
 - ❖ Projection should not ignore critical features

Proposed methods:

- ❖ Composition
 - ❖ Carvalho, et al. (2023) integrated multiple costs ‘unseen’ during training

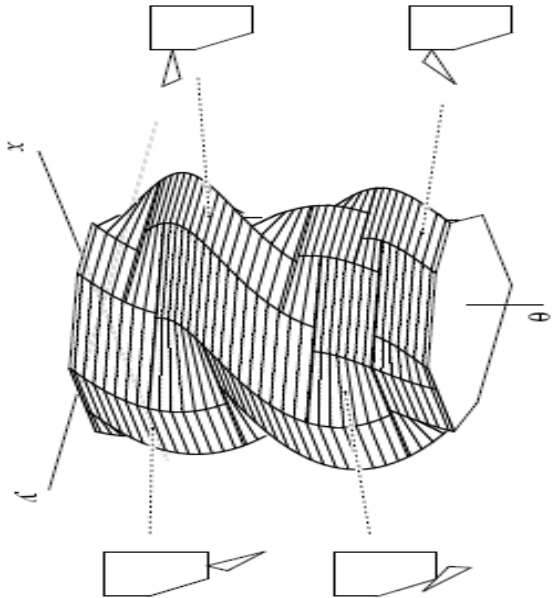
Open Problems – Gathering Expert Demonstrations

- Ichter, et al. 2019
 - ~100k in each environment, often human demonstrations were used
- Carvalho, et al. 2023
 - 10k trajectories, doubly smoothened and optimized in each environment
- ❖ LfH offers promising solution
 - ❖ Flexible hallucination methods would increase applicability
 - ❖ Reasoning about coverage of hallucinations, adapting to increase coverage
 - ❖ Develop exploration strategies to increase coverage of action space of input trajectories

Thank you

Supplemental – Config. Space

$SE(2)$



16-735, Howie Choset with slides from G.D. Hager, Z. Dodds, and Dinesh Mocha

Supplemental – Diffusion Math

$$\begin{aligned} \log p_{\theta}(\tau_{t-1}|\tau_t, t) &= \log \mathcal{N}(\tau_t; \mu_t = \mu_{\theta}(\tau_t, t), \Sigma_t) \\ &\propto -\frac{1}{2}(\tau_{t-1} - \mu_t)^{\top} \Sigma_t^{-1} (\tau_{t-1} - \mu_t). \end{aligned} \quad (5)$$

By definition of the noise schedule β_t , as the denoising step approaches zero, so does the noise covariance $\lim_{t \rightarrow 0} \|\Sigma_t\| = 0$. Therefore, $p_{\theta}(\tau_{t-1}|\tau_t, t)$ concentrates all the mass close to the mean μ_t , and therefore the task log-likelihood is approximated with a first-order Taylor expansion around μ_t

$$\log p(\mathcal{O}|\tau_{t-1}) \approx \log p(\mathcal{O}|\tau_{t-1} = \mu_t) + (\tau_{t-1} - \mu_t)g, \quad (6)$$

with $g = \nabla_{\tau_{t-1}} \log p(\mathcal{O}|\tau_{t-1})|_{\tau_{t-1}=\mu_t}$. Combining (5) and (6) we obtain

$$\begin{aligned} &\log p(\tau_{t-1}|\tau_t, t, \mathcal{O}) \\ &\propto -\frac{1}{2}(\tau_{t-1} - \mu_t)^{\top} \Sigma_t^{-1} (\tau_{t-1} - \mu_t) + (\tau_{t-1} - \mu_t)g \\ &\propto -\frac{1}{2}(\tau_{t-1} - \mu_t - \Sigma_t g)^{\top} \Sigma_t^{-1} (\tau_{t-1} - \mu_t - \Sigma_t g) \\ &= \log p(z), \text{ with } p(z) = \mathcal{N}(z; \mu_t + \Sigma_t g, \Sigma_t). \end{aligned}$$

Hence, sampling from the task-conditioned posterior is equivalent to sampling from a Gaussian distribution with mean and covariance as $p(z)$. In the motion planning-as-inference case, we have from (2)

$$g = \nabla_{\tau_{t-1}} \log p(\mathcal{O}|\tau_{t-1}) = -\sum_i \lambda_i \nabla_{\tau_{t-1}} c_i(\tau_{t-1}),$$

Algorithm 1: Motion Planning Diffusion

—TRAINING—

Input: Collision-free trajectories $\mathcal{D}$, Diffusion model ε_{θ} , learning rate α , noise schedule terms $\bar{\alpha}_t$

```

1 while training is not finished do
  ▷ sample a batch of trajectories
2   $\tau_0 \sim \mathcal{D}, \varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1, N)$ 
  ▷ compute the diffusion loss function
3   $\tau_t = \sqrt{\bar{\alpha}_t} \tau_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$ 
4   $\mathcal{L}(\theta) = \|\varepsilon - \varepsilon_{\theta}(\tau_t, t)\|_2^2$ 
  ▷ gradient update
5   $\theta = \theta + \alpha \nabla_{\theta} \mathcal{L}(\theta)$ 
```

—INFERENCE—

Input: Pre-trained diffusion model ε_{θ} , start and goal states (s_s, s_g) , motion planning costs c_i , temperatures λ_i , scheduling terms $(\alpha_t, \bar{\alpha}_t, \sigma_t)$

```

  ▷ sample a batch of trajectories
6   $\tau_N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
  ▷ hard set start and goal states
7   $\tau_N[0] = s_s, \tau_N[H-1] = s_g$ 
8  for  $t = N, \dots, 1$  do
  ▷ compute the diffusion prior mean
9   $\mu_t = \frac{1}{\sqrt{\alpha_t}} \left( \tau_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_{\theta}(\tau_t, t) \right)$ 
  ▷ compute weighted gradient of costs
10  $g = -\sum_i \lambda_i \nabla_{\tau_{t-1}} c_i(\tau_{t-1})$ 
  ▷ move trajectories to low cost regions
11  $\tau_{t-1} = \mu_t + g + \sigma_t z, \quad z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
12  $\tau_{t-1}[0] = s_s, \tau_{t-1}[H-1] = s_g$ 
```

Output: optimized batch of trajectories τ_0

$$q(\tau_t|\tau_{t-1}, t) = \mathcal{N}(\tau_t; \sqrt{1 - \beta_t} \tau_{t-1}, \beta_t \mathbf{I})$$

$$\mu_{\theta}(\tau_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\tau_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_{\theta}(\tau_t, t) \right),$$

Supplemental – Gen Model Taxonomy

